

223028000_MUSONERA JEAN DE DIEU

A1: Fragment & Recombine Main Fact

1. Table expense_A creation on Node A:

```
CREATE TABLE Expense_A (  
    Expense_ID INT PRIMARY KEY,  
    Description VARCHAR(100),  
    Amount DECIMAL(10,2)  
);
```

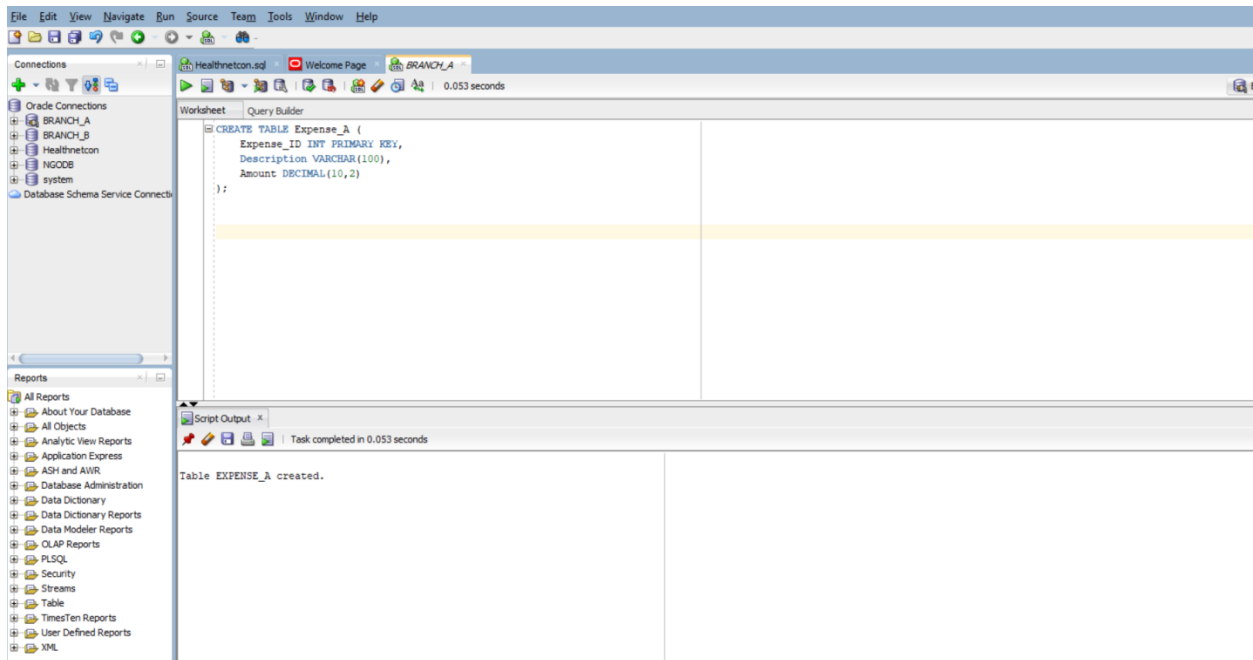
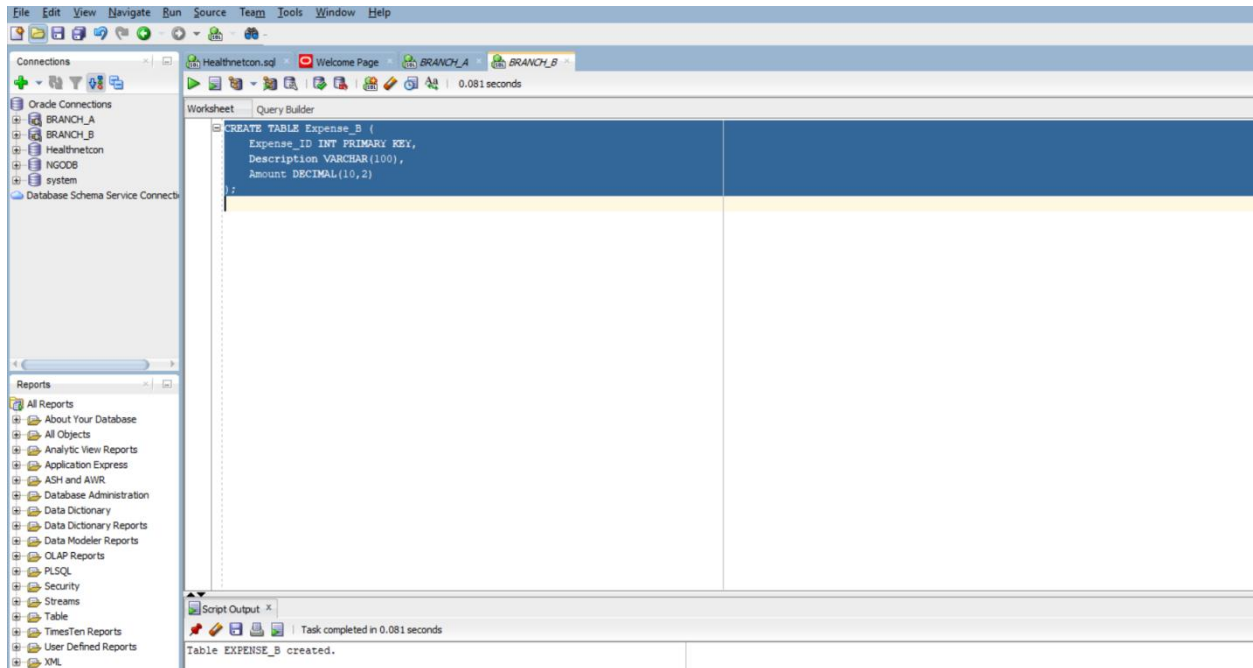


Table expense_B creation on Node B:

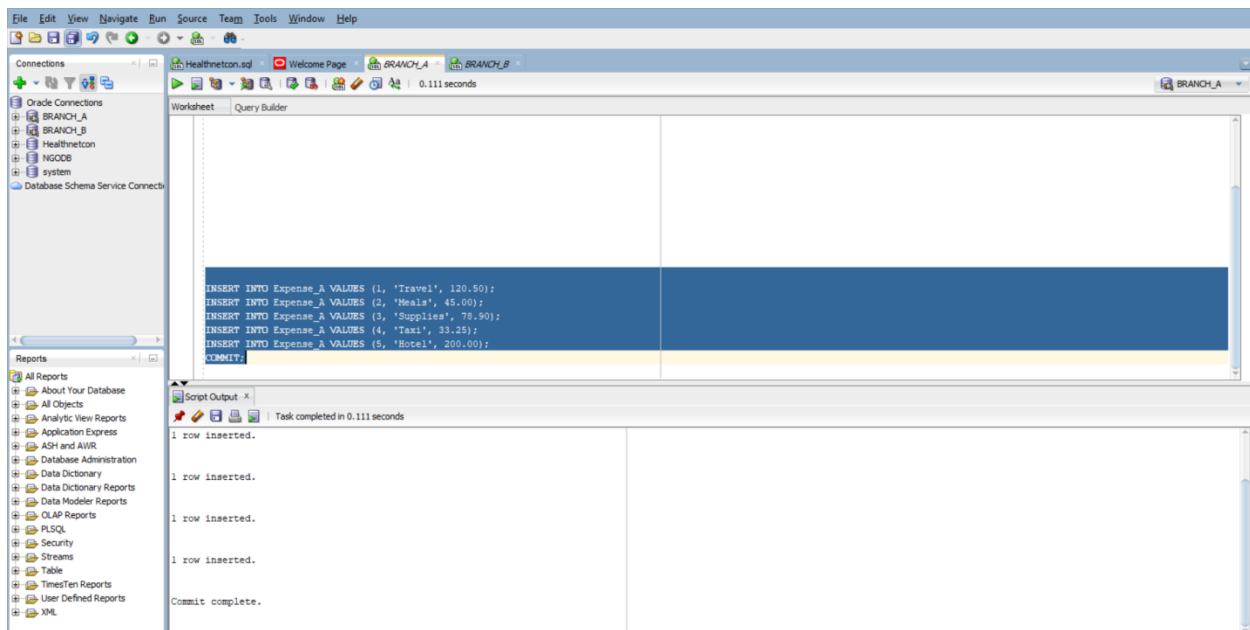
```
CREATE TABLE Expense_B (  
    Expense_ID INT PRIMARY KEY,  
    Description VARCHAR(100),  
    Amount DECIMAL(10,2)
```

);

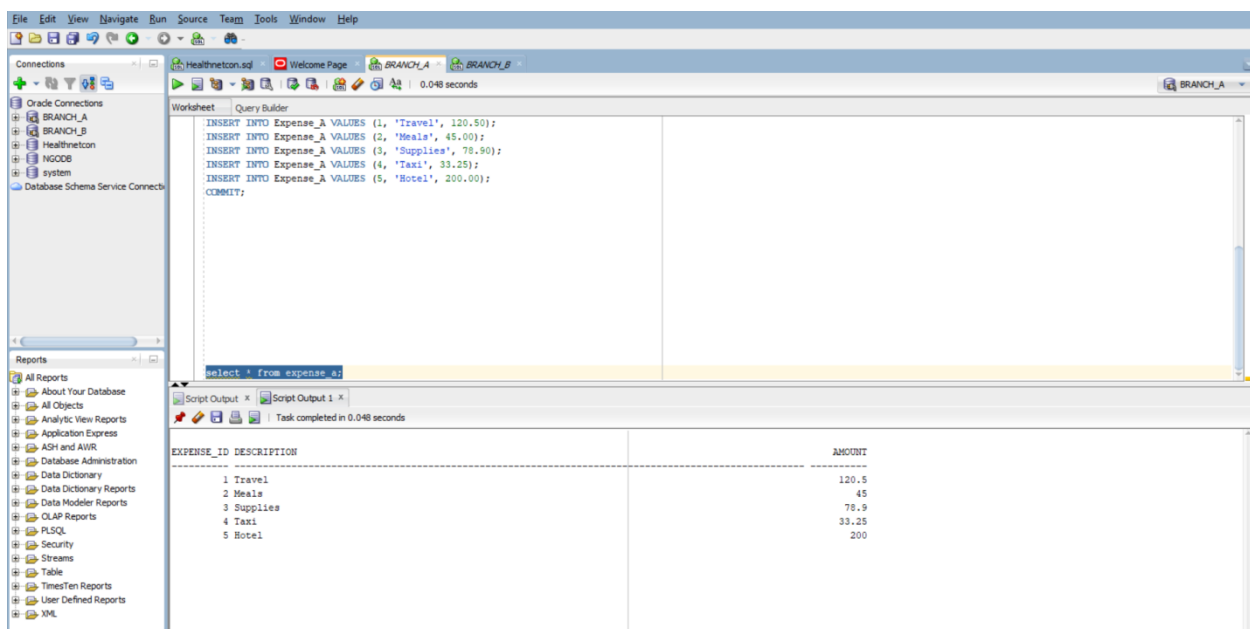


2. Inserting data in expense table on node A

```
INSERT INTO Expense_A VALUES (1, 'Travel', 120.50);  
INSERT INTO Expense_A VALUES (2, 'Meals', 45.00);  
INSERT INTO Expense_A VALUES (3, 'Supplies', 78.90);  
INSERT INTO Expense_A VALUES (4, 'Taxi', 33.25);  
INSERT INTO Expense_A VALUES (5, 'Hotel', 200.00);  
COMMIT;
```



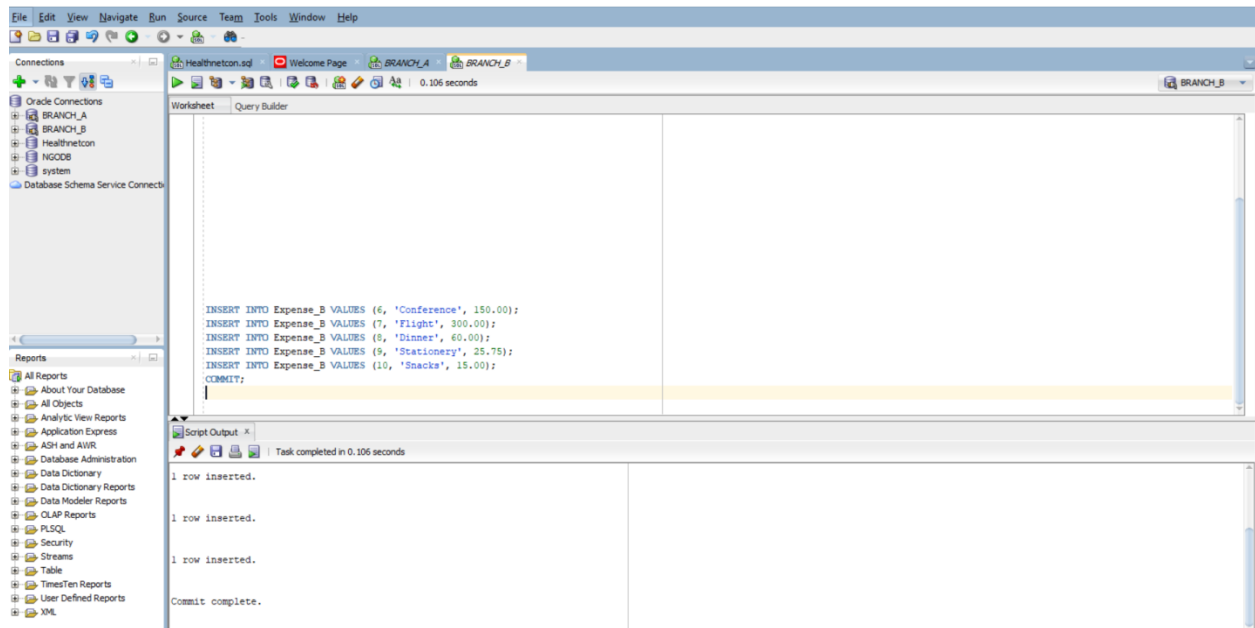
Checking Inserted data



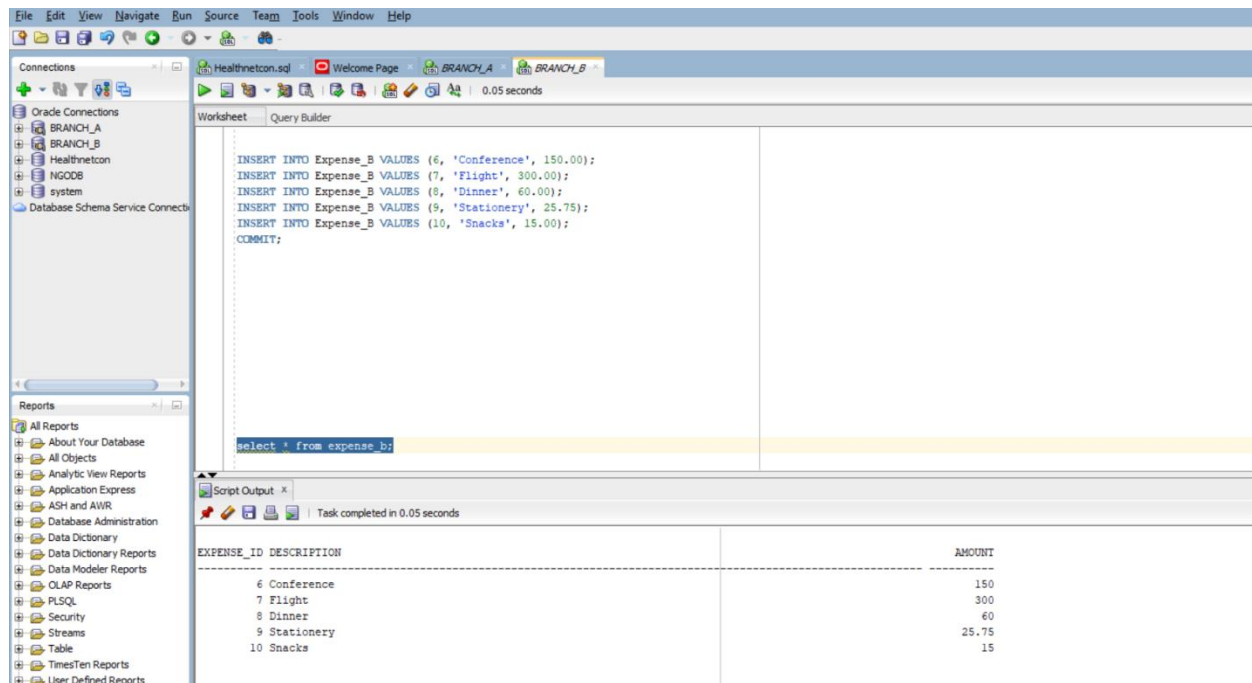
Inserting data in expense table on node B

INSERT INTO Expense_B VALUES (6, 'Conference', 150.00);

```
INSERT INTO Expense_B VALUES (7, 'Flight', 300.00);  
  
INSERT INTO Expense_B VALUES (8, 'Dinner', 60.00);  
  
INSERT INTO Expense_B VALUES (9, 'Stationery', 25.75);  
  
INSERT INTO Expense_B VALUES (10, 'Snacks', 15.00);  
  
COMMIT;
```



Checking inserted data



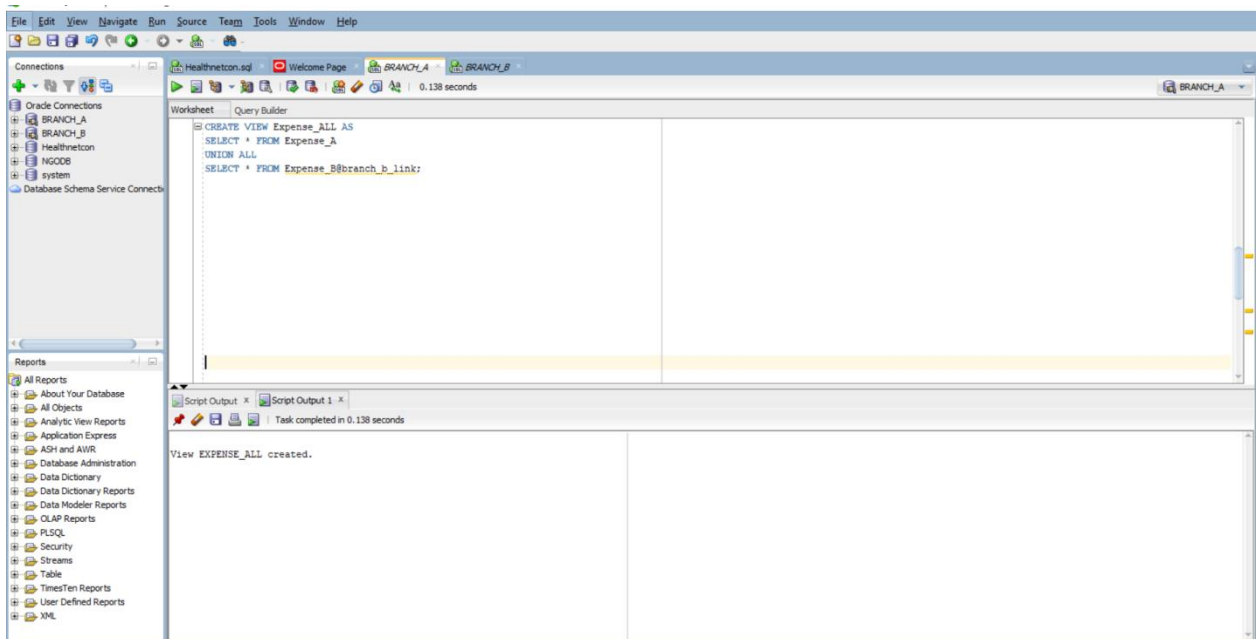
3.Create View on Node_A

CREATE VIEW Expense_ALL AS

SELECT * FROM Expense_A

UNION ALL

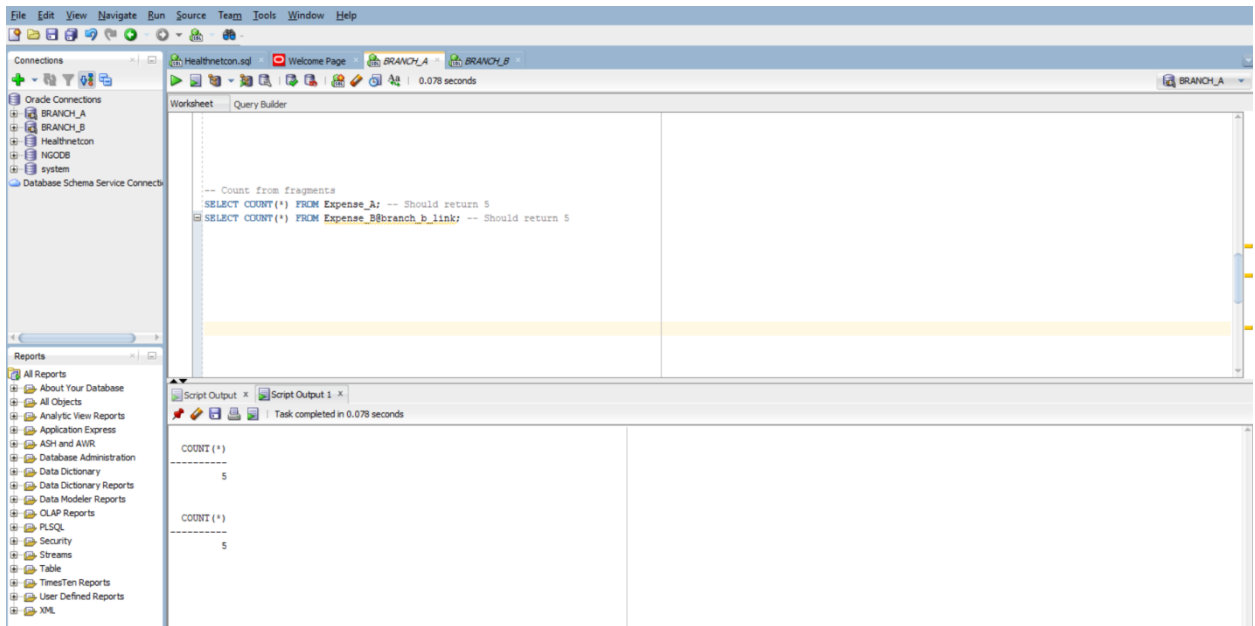
SELECT * FROM Expense_B@branch_b_link;



3. A. count from fragments

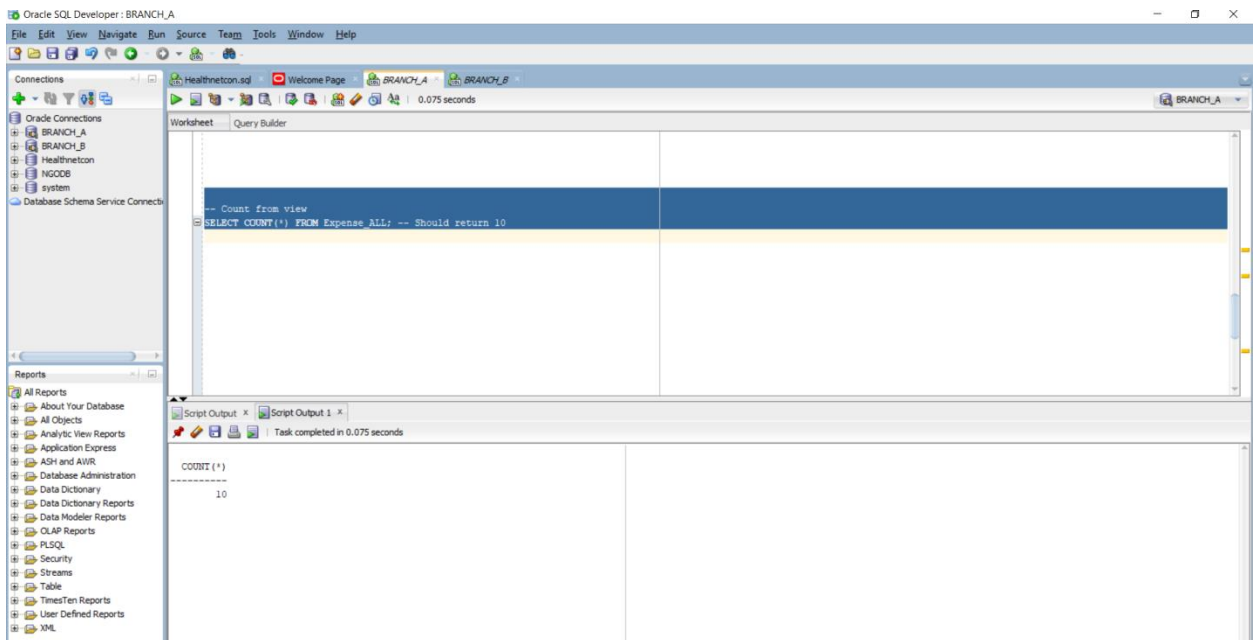
SELECT COUNT(*) FROM Expense_A; -- Should return 5

SELECT COUNT(*) FROM Expense_B@branch_b_link; -- Should return 5



B. count from View

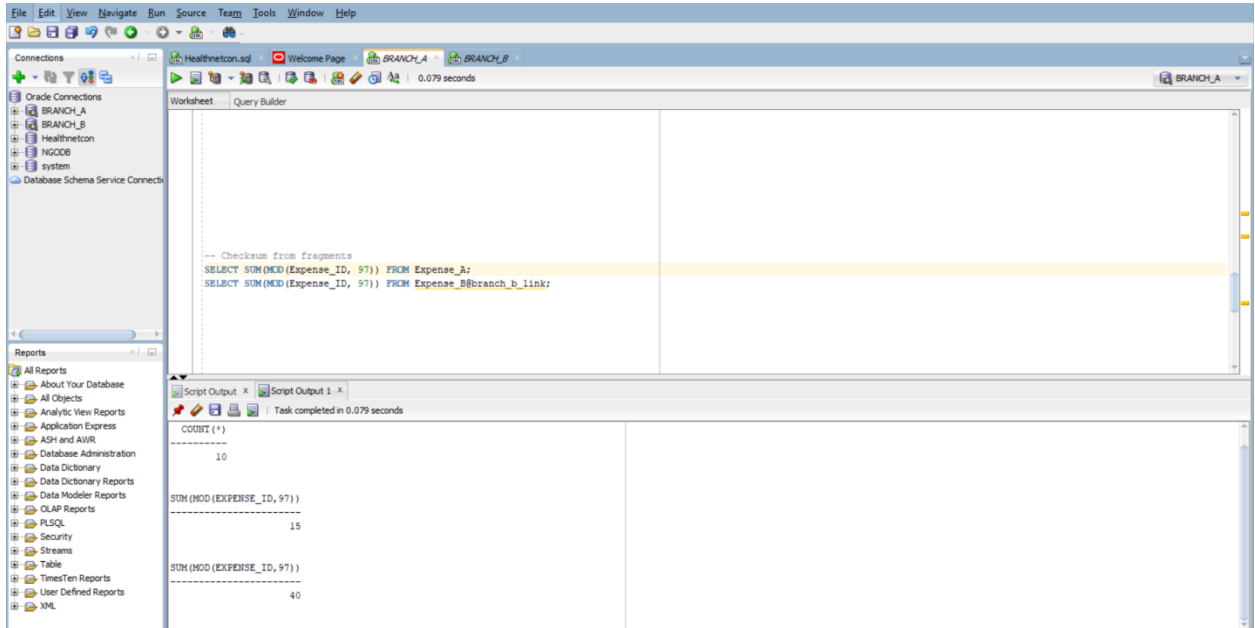
SELECT COUNT(*) FROM Expense_ALL;



c. Checksum from fragments

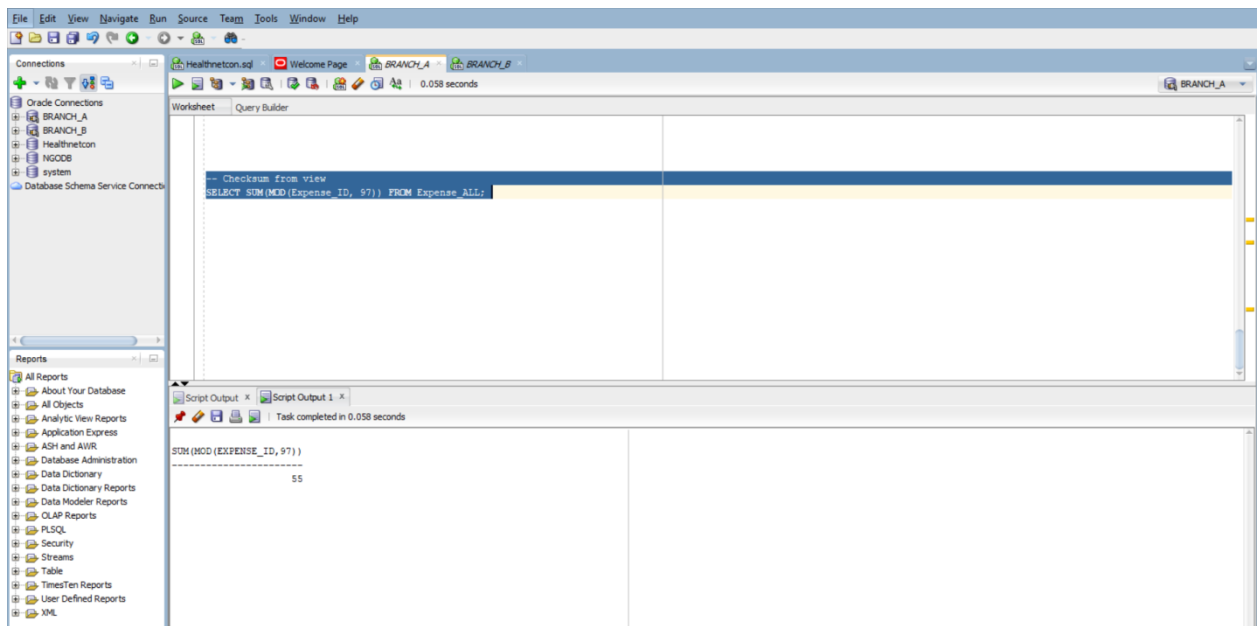
```
SELECT SUM(MOD(Expense_ID, 97)) FROM Expense_A;
```

```
SELECT SUM(MOD(Expense_ID, 97)) FROM Expense_B@branch_b_link;
```



d. Checksum from View

```
SELECT SUM(MOD(Expense_ID, 97)) FROM Expense_ALL;
```

A2 :Database Link & Cross-Node Join

a. creation of database link 'proj_link' to Node_B

```
CREATE DATABASE LINK proj_link
```

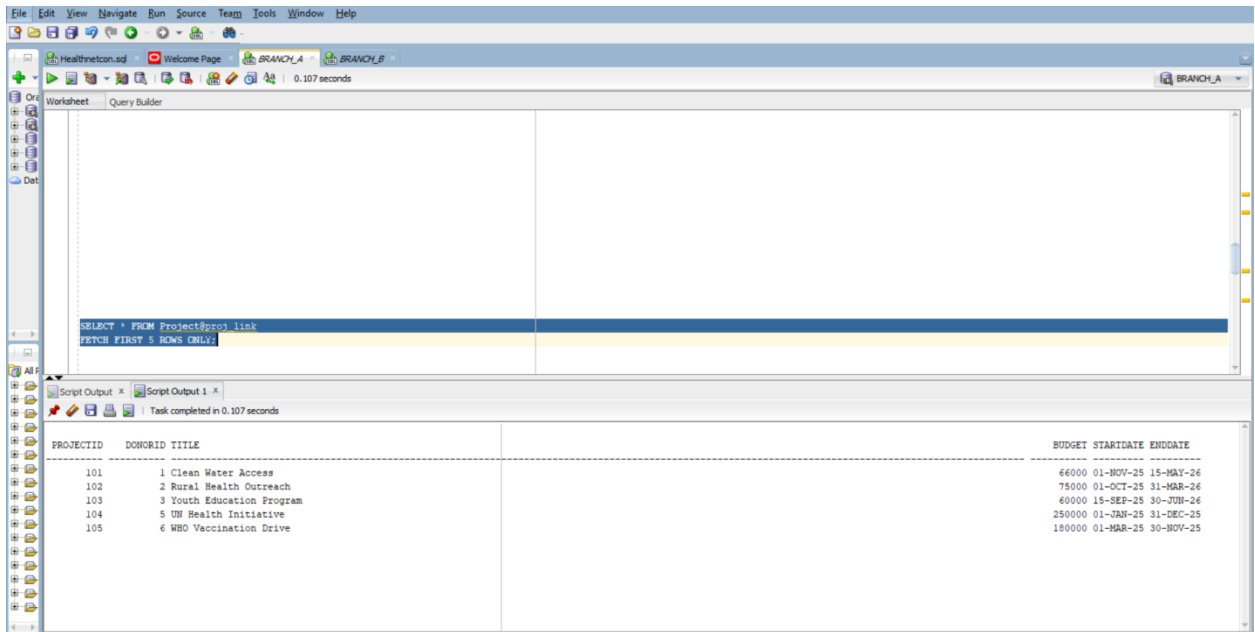
```
CONNECT TO branchdb_b IDENTIFIED BY "12345"
```

```
USING 'XEPDB1';
```

b. Remote SELECT on Project@proj_link showing up to 5 sample rows.

```
SELECT * FROM expense@proj_link
```

```
FETCH FIRST 5 ROWS ONLY;
```



c. Distributed join:

```
SELECT e.Expense_ID, e.Description, e.Amount, d.Donor_Name, d.Donation_Amount
FROM Expense_A e
CROSS JOIN branchdb_b.Donor@proj_link d
WHERE e.Amount > 100
      AND d.Donation_Amount >= 500
      AND d.Donation_Date >= DATE '2025-01-01'
FETCH FIRST 10 ROWS ONLY;
```

The screenshot shows a SQL IDE with a query editor and a results pane. The query is as follows:

```

SELECT e.Expense_ID, e.Description, e.Amount, d.Donor_Name, d.Donation_Amount
FROM Expense_A a
CROSS JOIN branchdb_b.Donor@proj_link d
WHERE e.Amount > 100
AND d.Donation_Amount >= 500
AND d.Donation_Date >= DATE '2025-01-01'
FETCH FIRST 10 ROWS ONLY;

```

The results pane shows the following data:

EXPENSE_ID	DESCRIPTION	AMOUNT	DONOR_NAME
1	Travel	120.5	Alice Johnson
5	Hotel	200	Alice Johnson
1	Travel	120.5	Bob Smith
5	Hotel	200	Bob Smith
1	Travel	120.5	David Kim
5	Hotel	200	David Kim

6 rows selected.

A3 :Parallel vs Serial Aggregation

SERIAL Aggregation on **Expense_ALL**:

SELECT Description, SUM(Amount) AS Total_Amount

FROM Expense_ALL

GROUP BY Description;

The screenshot shows a SQL IDE interface with a menu bar (File, Edit, View, Navigate, Run, Source, Team, Tools, Window, Help) and a toolbar. The main window displays a SQL query in the 'Query Builder' tab:

```
SELECT Description, SUM(Amount) AS Total_Amount
FROM Expense_ALL
GROUP BY Description;
```

Below the query, the 'Script Output' tab shows the results of the query, indicating 'Task completed in 0.052 seconds'. The results are displayed in a table with 10 rows selected.

Description	Total_Amount
Travel	120.5
Meals	45
Supplies	78.9
Taxi	33.25
Hotel	200
Conference	150
Flight	300
Dinner	60
Stationery	25.75
Snacks	15

10 rows selected.

EXPLAIN PLAN FOR

SELECT Description, SUM(Amount) AS Total_Amount

FROM Expense_ALL

GROUP BY Description;

SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);

Healthnetcon.sql | Welcome Page | BRANCH_A | BRANCH_B | 0.30599999 seconds | BRANCH_A

Worksheet | Query Builder

```

SELECT Description, SUM(Amount) AS Total_Amount
FROM Expense_ALL
GROUP BY Description;

SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);

```

Script Output x | Script Output 1 x | Task completed in 0.306 seconds

PLAN_TABLE_OUTPUT

Plan hash value: 1553133699

Id	Operation	Name	Rows	Bytes	Cost (CPU)	Time	Inst	IN-OUT
0	SELECT STATEMENT		87	5655	6 (17)	00:00:01		
1	HASH GROUP BY		87	5655	6 (17)	00:00:01		
2	VIEW	EXPENSE_ALL	87	5655	5 (0)	00:00:01		
3	UNION-ALL							
4	TABLE ACCESS FULL	EXPENSE_A	5	325	3 (0)	00:00:01		
5	REMOTE	EXPENSE_B	82	5330	2 (0)	00:00:01	BRANC-	R->S

PLAN_TABLE_OUTPUT

Remote SQL Information (identified by operation id):

```

5 - SELECT "DESCRIPTION","AMOUNT" FROM "EXPENSE_B" "EXPENSE_B" (accessing
    'BRANCH_B_LINK' )

```

Note

PARALLEL Aggregation

```
SELECT /*+ PARALLEL(Expense_A,8) PARALLEL(Expense_B,8) */
```

```
    Description, SUM(Amount) AS Total_Amount
```

```
FROM Expense_ALL
```

```
GROUP BY Description;
```

The screenshot shows a SQL query execution interface. The top part displays the query in the Query Builder:

```
SELECT /*+ PARALLEL(Expense_A,8) PARALLEL(Expense_B,8) */
      Description, SUM(Amount) AS Total_Amount
FROM Expense_ALL
GROUP BY Description;
```

The bottom part shows the results of the query in a table:

Description	Total_Amount
Travel	120.5
Meals	45
Supplies	78.9
Taxi	33.25
Hotel	200
Conference	150
Flight	300
Dinner	60
Stationery	25.75
Snacks	15

10 rows selected.

EXPLAIN PLAN FOR

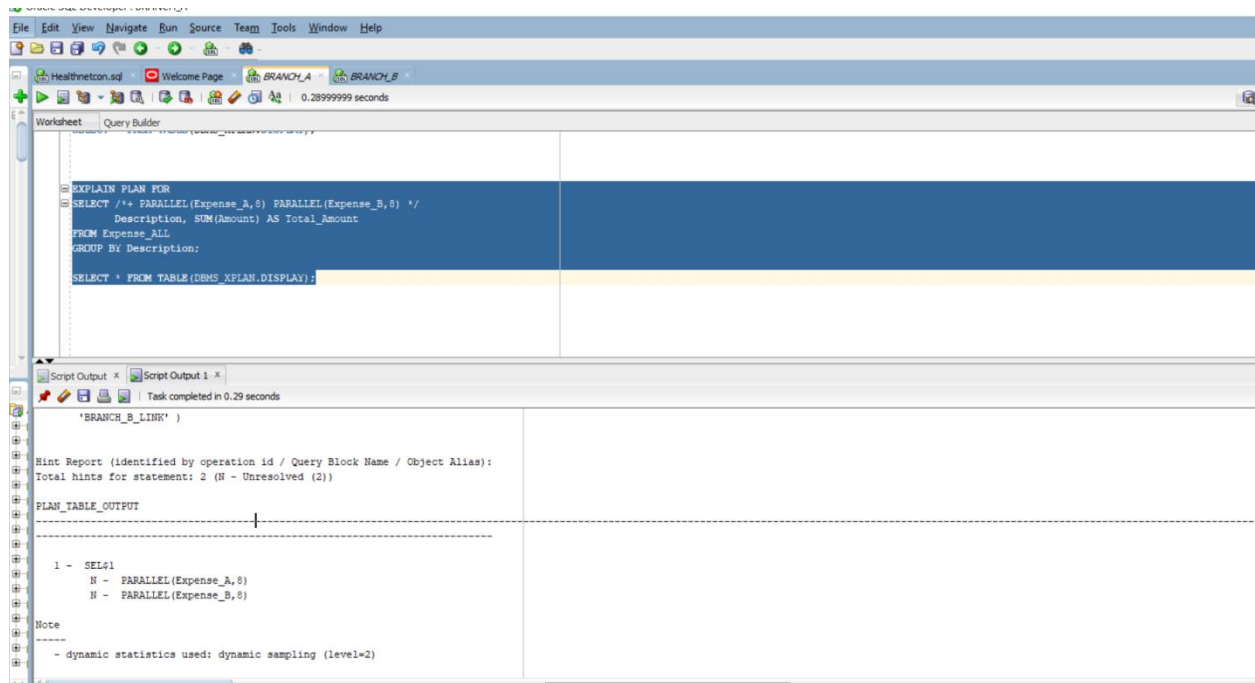
SELECT /*+ PARALLEL(Expense_A,8) PARALLEL(Expense_B,8) */

Description, SUM(Amount) AS Total_Amount

FROM Expense_ALL

GROUP BY Description;

SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);



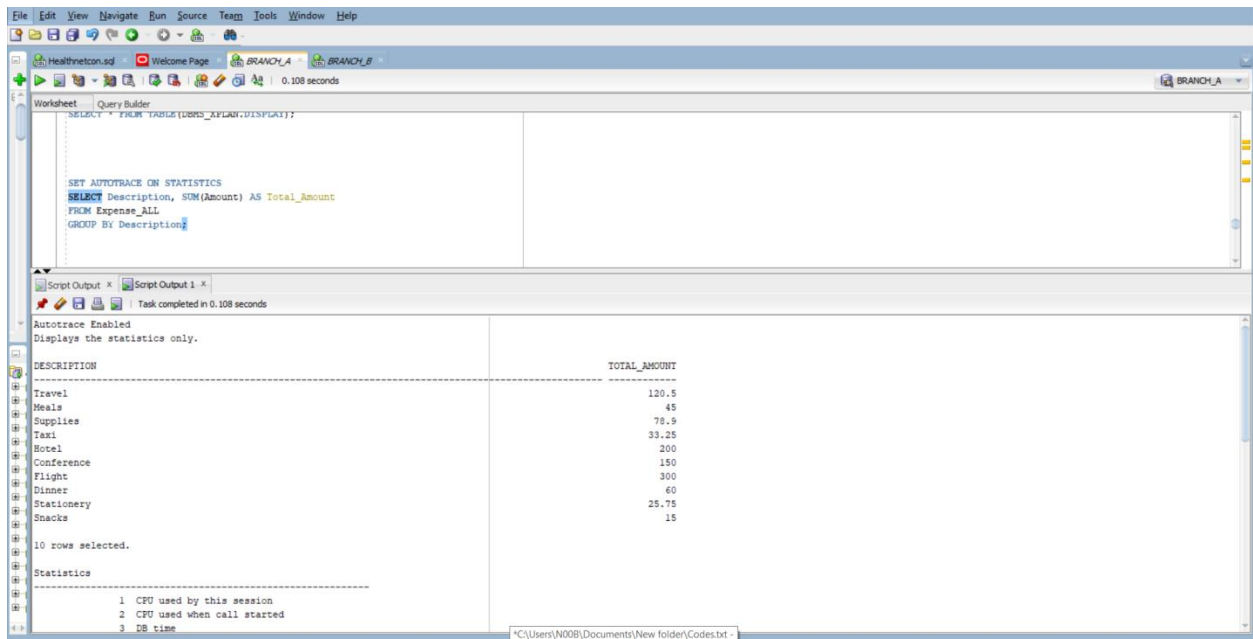
c. Serial Autotrace

SET AUTOTRACE ON STATISTICS

SELECT Description, SUM(Amount) AS Total_Amount

FROM Expense_ALL

GROUP BY Description;



Paerallel Autotrace

SET AUTOTRACE ON STATISTICS

SELECT /*+ PARALLEL(Expense_A,8) PARALLEL(Expense_B,8) */

Description, SUM(Amount) AS Total_Amount

FROM Expense_ALL

GROUP BY Description;

The screenshot shows the Oracle SQL Developer interface. The query editor contains the following SQL code:

```

SET AUTOTRACE ON STATISTICS
SELECT /*+ PARALLEL(Expense_A,8) PARALLEL(Expense_B,8) */
      Description, SUM(Amount) AS Total_Amount
FROM Expense_ALL
GROUP BY Description;

```

The Script Output window shows the execution results:

```

Displays the statistics only.

DESCRIPTION                                TOTAL_AMOUNT
-----
Travel                                     120.5
Meals                                       45
Supplies                                  78.9
Taxi                                       33.25
Hotel                                     200
Conference                                150
Flight                                    300
Dinner                                    60
Stationery                                25.75
Snacks                                    15
10 rows selected.

Statistics
-----
2 CPU used by this session
2 CPU used when call started
3 DB time
2 Requests to/from client

```

Mode	Execution Plan Notes
Serial	Performs a full table scan on Expense_ALL using a single-threaded aggregation. Minimal CPU and I/O usage. Suitable for small datasets.
Parallel	Executes parallel full scans on Expense_A and Expense_B using multiple PX slaves. Includes PX COORDINATOR, PX SEND, and PX RECEIVE operations. Higher resource allocation, but similar timing due to small data volume.

A4 :Two-Phase Commit & Recovery

- PL/SQL block that inserts ONE local row (related to Expense) on Node_A and ONE remote row into Expense@proj_link

BEGIN

-- Insert into local Expense_A on Node_A

INSERT INTO Expense_A (Expense_ID, Description, Amount)

```
VALUES (9001, 'Test Local Expense', 123.45);
```

```
-- Insert into remote Expense on Node_B via proj_link
```

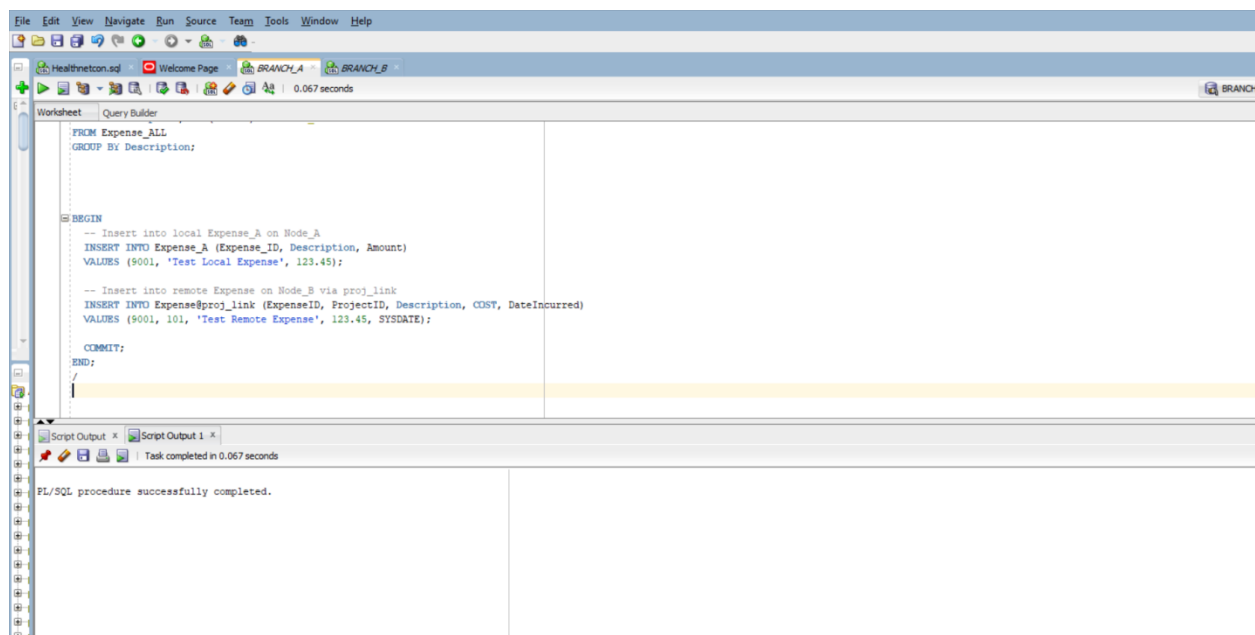
```
INSERT INTO Expense@proj_link (ExpenseID, ProjectID, Description, COST, DateIncurred)
```

```
VALUES (9001, 101, 'Test Remote Expense', 123.45, SYSDATE);
```

```
COMMIT;
```

```
END;
```

```
/
```



b. failure is not possible Oracle, whether remkote or local transaction

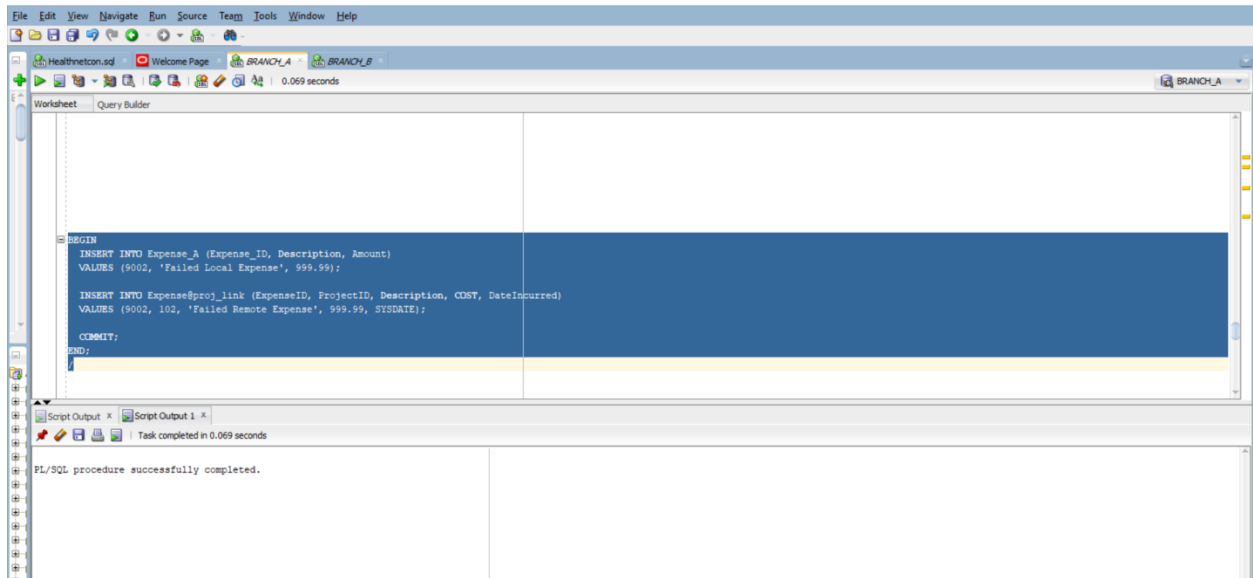
```
BEGIN
```

```
INSERT INTO Expense_A (Expense_ID, Description, Amount)
```

```
VALUES (9002, 'Failed Local Expense', 999.99);
```

```
INSERT INTO Expense@proj_link (ExpenseID, ProjectID, Description, COST,
DateIncurred)
VALUES (9002, 102, 'Failed Remote Expense', 999.99, SYSDATE);

COMMIT;
END;
/
```



C.

It return no rows beacause the transaction went through: Query DBA_2PC_PENDING

```
SELECT LOCAL_TRAN_ID, GLOBAL_TRAN_ID, STATE
FROM DBA_2PC_PENDING;
```

File Edit View Navigate Run Source Team Tools Window Help

Healthnetcon.sql Welcome Page BRANCH_A BRANCH_B 0.064 seconds

Worksheet Query Builder

```

BEGIN
  INSERT INTO Expense_A (Expense_ID, Description, Amount)
  VALUES (9002, 'Failed Local Expense', 999.99);

  INSERT INTO Expense@proj_link (ExpenseID, ProjectID, Description, COST, DateIncurred)
  VALUES (9002, 102, 'Failed Remote Expense', 999.99, SYSDATE);

  COMMIT;
END;
/

SELECT LOCAL_TRAN_ID, GLOBAL_TRAN_ID, STATE
FROM DBA_2PC_PENDING;

```

Script Output x Script Output 1 x

Task completed in 0.064 seconds

No rows selected

Statistics

2	Requests to/from client
2	SQL*Net roundtrips to/from client
2	buffer is not pinned count
340	bytes received via SQL*Net from client
60849	bytes sent via SQL*Net to client
2	calls to get snapshot scn: kcmgss
2	calls to kcmgcs
2	execute count
9	non-idle wait count
2	opened cursors cumulative

d. No need to roll back since no transaction is hanging

e. inserted row

File Edit View Navigate Run Source Team Tools Window Help

Healthnetcon.sql Welcome Page BRANCH_A BRANCH_B 0.061 seconds

Worksheet Query Builder

```

VALUES (9002, 102, 'Failed Remote Expense', 999.99, SYSDATE);

COMMIT;
END;
/

SELECT LOCAL_TRAN_ID, GLOBAL_TRAN_ID, STATE
FROM DBA_2PC_PENDING;

SELECT COUNT(*) AS Local_Count
FROM Expense_A
WHERE Expense_ID = 9002;

```

Script Output x Script Output 1 x

Task completed in 0.061 seconds

LOCAL_COUNT

1

Statistics

2	CPU used by this session
2	CPU used when call started
2	DB time
2	Requests to/from client
2	SQL*Net roundtrips to/from client
1	buffer is not pinned count
345	bytes received via SQL*Net from client

A5 :Distributed Lock Conflict & Diagnosis

- a. Open Session 1 on Node_A:UPDATE a single row in project

BEGIN

UPDATE Project

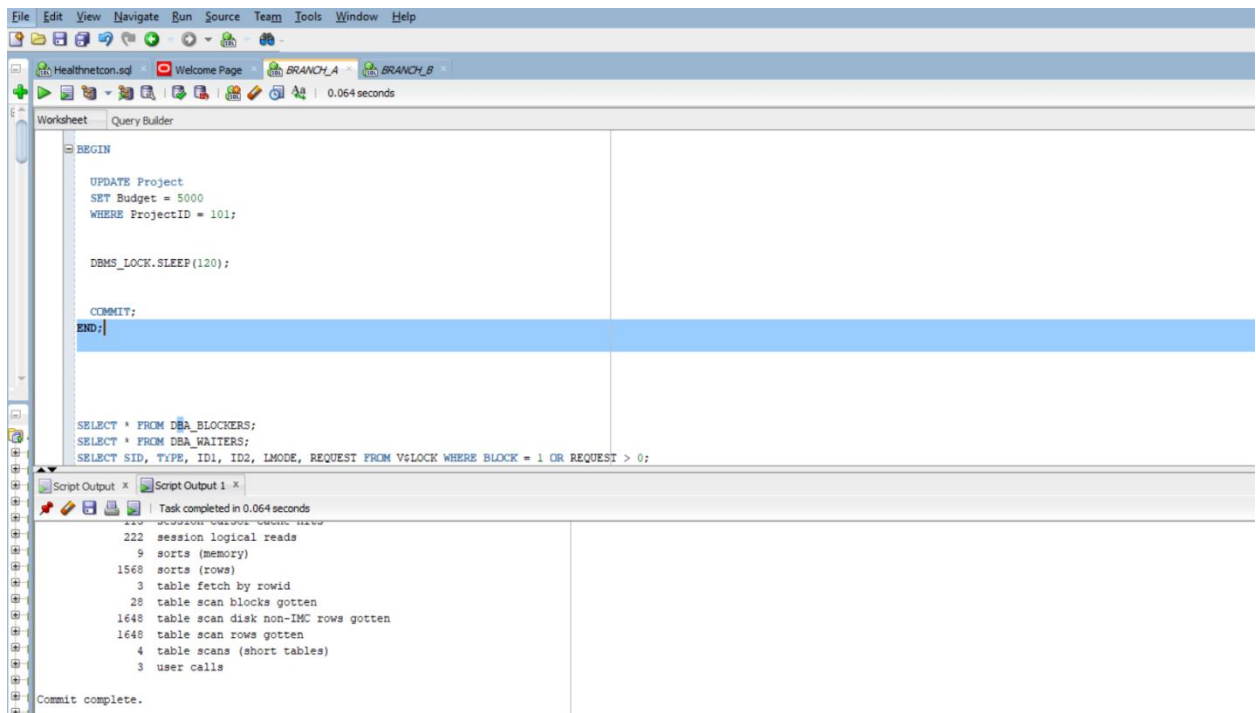
SET Budget = 5000

WHERE ProjectID = 101;

DBMS_LOCK.SLEEP(120);

COMMIT;

END;



b. Open Session 2on Node_B:UPDATE remotely a single row in Project

BEGIN

UPDATE Project@BRANCH_A_LINK

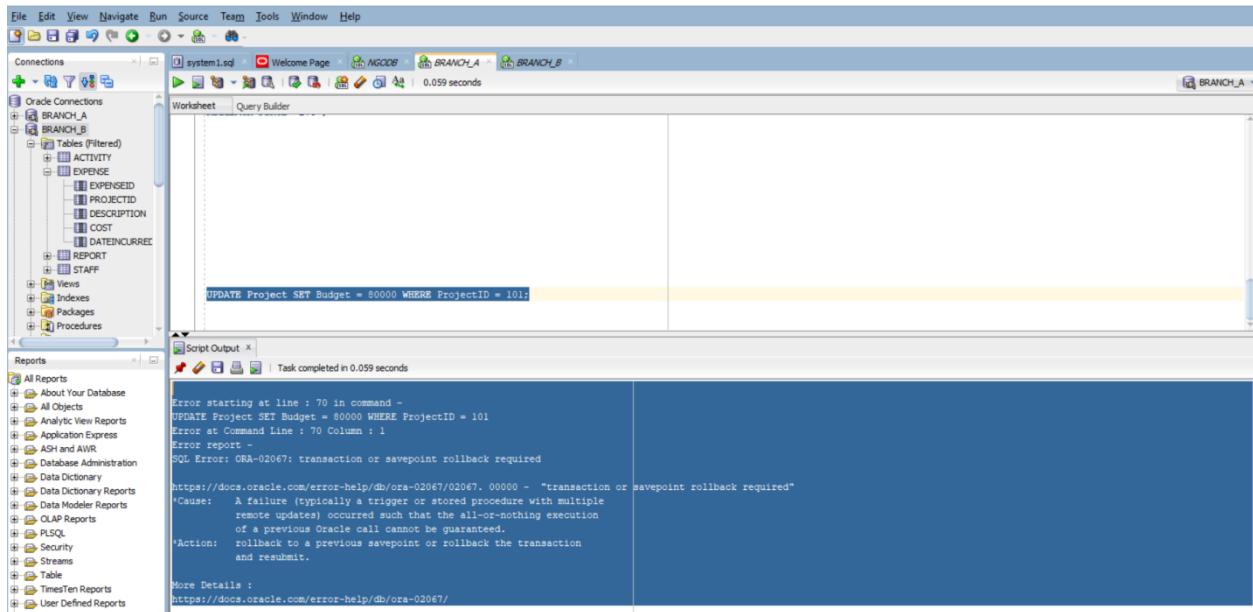
SET Budget = 5000

WHERE ProjectID = 101;

DBMS_LOCK.SLEEP(120);

COMMIT;

END;



d. View Locks

```
SELECT SID, TYPE, ID1, ID2, LMODE, REQUEST, BLOCK
FROM V$LOCK
WHERE BLOCK = 1 OR REQUEST > 0;
```

Oracle SQL Developer interface showing a query execution. The query is:

```
SELECT
  s.sid,
  s.serial#,
  s.username,
  l.type,
  l.lmode,
  l.request,
  l.block
FROM
  V$LOCK l
JOIN
  V$SESSION s ON l.sid = s.sid
WHERE
  l.type = 'TX'; -- Transaction locks
```

The script output shows the task completed in 0.098 seconds and displays the following results:

SID	SERIAL#	USERNAME	TY	LMODE	REQUEST	BLOCK
28	3170	BRANCHEDB_A	TX	0	6	0
395	65340	BRANCHEDB_B	TX	6	0	1

e. Release the lock

Oracle SQL Developer interface showing a query execution. The query is:

```
-- Show blocking sessions
```

The script output shows the task completed in 0.046 seconds and displays the following results:

```
2 recursive cpu usage
1 rows fetched via callback
-2 session cursor cache count
34 session cursor cache hits
59 session logical reads
1 sorts (memory)
1545 sorts (rows)
2 table fetch by rowid
3 user calls
Commit complete.
```